

Assessment: Jeopardy

 [jeopardy-section1-starter-code.zip](#) 3316.2KB

Assessments

Remember, assessments are meant to be completed **by you**, not as a shared exercise with friends or other members of your cohort.

All code submitted should be **written by you**. If you incorporate code from elsewhere, it must be properly attributed.

Please do not put your assessment on GitHub.

It's Jeopardy!

Using jQuery and AJAX, you'll build a small, straightforward Jeopardy game.

You will be using the API listed here: <https://github.com/rithmschool/jeopardy-api>

Click here to find a working version: <https://jeopardy-demo.surge.sh/>

Please note that the API link has been updated; the jservice API students have been using till 2023 is not functional anymore. The new API is fairly simple to understand; please go through the following instructions to know more.

The API

In order to fetch the correct data you will be using an external **API**. The "endpoint" is <https://rithm-jeopardy.herokuapp.com/api>.

We will need two following subsequent endpoints:

1. **GET** "[https://rithm-jeopardy.herokuapp.com/api/categories?count=\[integer\]](https://rithm-jeopardy.herokuapp.com/api/categories?count=[integer])"
2. **GET** "[https://rithm-jeopardy.herokuapp.com/api/category?id=\[integer\]](https://rithm-jeopardy.herokuapp.com/api/category?id=[integer])"

Retrieving Category Data

Firstly, we will need to retrieve **Category** data for our questions. The format for the **URL** is the following:

GET "[https://rithm-jeopardy.herokuapp.com/api/categories?count=\[integer\]](https://rithm-jeopardy.herokuapp.com/api/categories?count=[integer])"

To review our concepts. The **API** documentation is telling us that we can plug in a number in the **count=[integer]** part of the URL and the API will fetch us the data which we need.

The **GET** request will fetch us the following data:

```
let res = await axios.get(`https://rithm-jeopardy.herokuapp.com/api/categories?
```

```
count=100`)
```

```
// NOTICE WE HAVE PROVIDED THE VALUE 100 FOR THE <count> query, what happens  
if the value is changed? //
```

```
// what is in our res variable? //
```

```
// checking out res.data will produce the following result //
```

```
[
```

```
{  
  
  "id": 2,  
  
  "title": "baseball",  
  
  "clues_count": 5  
  
},  
  
{  
  
  "id": 3,  
  
  "title": "odd jobs",  
  
  "clues_count": 5  
  
},  
  
{  
  
  "id": 4,  
  
  "title": "movies",  
  
  "clues_count": 5  
  
},  
  
{  
  
  "id": 6,  
  
  "title": "\"cat\" egory",  
  
  "clues_count": 5  
  
},  
  
// and so on until 100 objects in an array  
  
]
```

Retrieving Questions in a Specific Category

Now that you know how to retrieve all available categories, think about how you can retrieve questions in a specific category:

To review, the format for the URL is the following:

GET `"https://rithm-jeopardy.herokuapp.com/api/category?id=[**integer**]"`

Just like in the previous example the **API** is telling us that if we plug in a number in the **id=integer** part of the **URL** we will receive an appropriate response.

For example:

```
let res = await axios.get(`https://rithm-jeopardy.herokuapp.com/api.... What should  
our URL look like here ... ?`)
```

```
// HINT: Look at the example above //
```

```
// what is in our res variable? //
```

```
// what will our result be? //
```

```
// Good luck! //
```

```
};
```

Working Further with the API

Before you begin the project, explore the **API**. What happens when you limit the responses in the first **URL**? (i.e 'count=5') or if you try to **GET** a Category which does NOT exist (i.e "id=0").

Note:

This is not the original Jeopardy API from the JService.io platform. The original one had a lot more categories and some 200K questions. However, that portal went down. We worked around it using an alternate API hosted by rithmschool. So, unlike the original API, this API has only a limited number of categories and questions in each category. However, it's more than sufficient to use it and build a front end application.

Requirements

- The game board should be 6 categories across, 5 question down, displayed in a table. Above this should be a header row with the name of each category.
- At the start of the game, you should randomly pick 6 categories from the jService API. For each category, you should randomly select 5 questions for that category.
- Initially, the board should show with ? on each spot on the board (on the real TV show, it shows dollar amount, but we won't implement this).
- When the user clicks on a clue ?, it should replace that with the question text.
- When the user clicks on a visible question on the board, it should change to the answer (if they click on a visible answer, nothing should happen)
- When the user clicks the "Restart" button at the bottom of the page, it should load new categories and questions.

We've provided an HTML file and CSS for the application (you shouldn't change the HTML file; if you want to tweak any CSS things, feel free to).

We've also provided a starter JS file with function definitions. Implement these functions to meet the required functionality.



Randomly picking multiple things

In the requirements, we've asked for 6 random categories. Unfortunately, the `jService` API doesn't have a method that returns a random category — you'll need to figure this out.

There are a few possible strategies here: